KG 구축 how 보완
자료 준비 · 구조화 자료 매핑 · 비정형 텍스트 추출

벤치마킹한 문헌의 원리와 우리 구현 과정, 그리고 다음 주 계획

2026.10.30 · 문경민

1. KG 구축 단계와 방법론 구체화 현황

- KG 구축은 아래 순서로 수행했다. 모든 단계의 수행은 끝났고, 이번 자료는 그중 자료 준비, 구조화 자료 매핑, 비정형 텍스트 추출의 방법론을 정리했다.
- 이 세 단계를 마치면 KG 에 들어갈 Entity 264 개와 속성 616 개가 만들어진다. 이들을 관계로 이어 KG 를 완성하는 것은 개체 해소·연결이다.
- 모든 단계를 수행했지만, 방법론을 정리한 것은 자료 준비부터 비정형 텍스트 추출까지이다. 나머지는 다음 주에 정리한다(6 장). 각 원리의 자세한 설명은 부록 E 에 있다.

표 1. 단계별 하는 일, 따른 원리, 방법론 정리 여부

단계	하는 일	따른 원리	방법론 정리
자료 준비	소스 구분과 전처리	소스 구분: Huang 외(2026)가 자료를 정형 표와 비정형 텍스트로 나누어 서로 다른 Class 를 채운 원리 전처리: 문헌 근거 없음	**정리함 (2 장)**
구조화 자료 매핑	정형 표의 행·열을 Entity 와 속성으로	Zhou 외(2023)가 OntoDSMS 구축에 사용한 D2RQ 의 매핑 원리	**정리함 (3 장)**
비정형 텍스트 추출	비고에서 운영행위, 별표 3 에서 운영기준을 뽑음	Huang 외(2026)의 규칙 기반 지식 추출	**정리함 (4 장)**
개체 해소·연결	서로 다른 자료의 Entity 를 관계로 이음	Purohit 외(2024)의 개체 중의성 해소 (GraphAide)	**아직 정리 못 함 (다음 주)**
검증	만든 KG 가 온톨로지 제약에 맞는지 확인	—	**아직 정리 못 함 (다음 주)**
저장	CSV 를 Cypher 로 바꿔 Neo4j 에 넣음	Huang 외(2026)의 저장 방식	**아직 정리 못 함 (다음 주)**
검색·답변	질문에 맞는 근거를 찾아 LLM 이 답함	—	**아직 정리 못 함 (다음 주)**

2. 자료 준비: 소스 구분과 전처리

2.1 소스 구분: 자료마다 처리 방법과 채우는 Class 를 정한다

- 따른 원리: Huang 외(2026)는 댐 결함 지식그래프(DDKG)를 만들 때 플랫폼의 정형 표로 c1~c4(Dam 등)를, 보고서의 비정형 텍스트로 c5~c7(위험 형태 등)을 채운다. Class 마다 출처를 정하고, 정형 표인지 비정형 텍스트인지에 따라 처리를 나눈다(부록 E.1).
- 우리 자료 5 가지도 같은 원리로 나누었다(그림 1). 승인 기록 CSV 와 K-water 일자료·시간자료는 열이 정해진 정형 표라서 열 값을 그대로 옮긴다(3 장 구조화 자료 매핑).
- 승인 기록의 비고 열은 표 안에 있지만 "(실제방류는 20 시부터)" 같은 문장이고, 규정 별표 3 은 선 문자(|, –)로 그린 글자라서 칸을 그대로 읽을 수 없다. 둘 다 글자를 읽어 값을 뽑아야 하므로 비정형으로 나누었다(4 장 비정형 텍스트 추출).
- 그래서 비고 열은 두 번 쓰인다. 구조화 자료 매핑에서는 승인 표의 한 칸으로서 원문을 그대로 옮기고, 비정형 텍스트 추출에서는 그 문장을 읽어 운영행위(Operation)를 뽑는다.

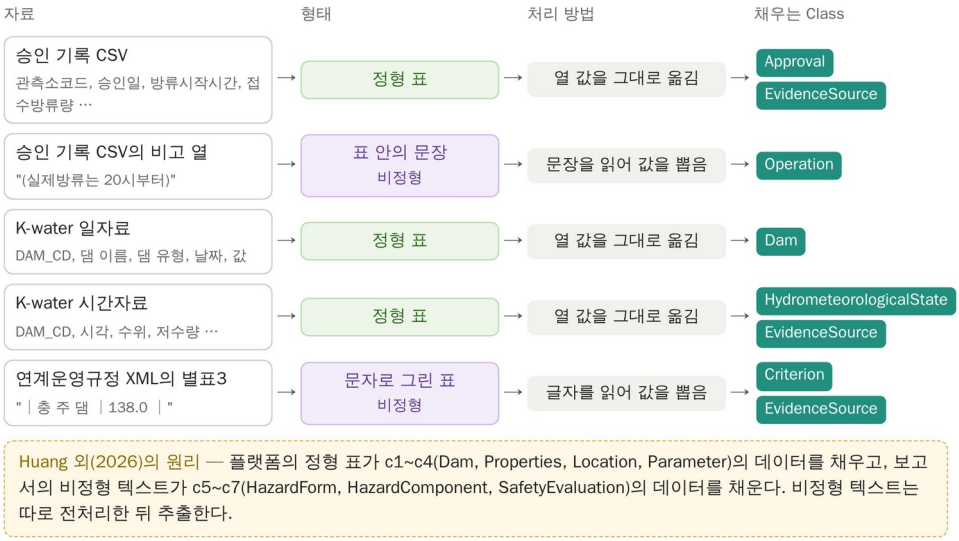


그림 1. 소스 구분 — 자료 5 가지의 형태, 처리 방법, 채우는 Class

2.2 전처리: 원본은 그대로 두고, 쓸 수 있는 형태로 정리한다

- Huang 외(2026)의 전처리는 형식이 제각각인 보고서를 마크다운으로 바꾸고(이미지는 OCR) 정규식으로 필요한 구간만 잘라내는 것이다. 우리 자료는 정형 표이거나 이미 글자(XML)라서 이 기술을 쓰지 않았다. 따라서 전처리는 문헌을 따른 방법이 아니라 자료 준비이다.
- 원칙은 원본 값은 바꾸지 않고, 지우거나 바꾼 것은 기록으로 남기는 것이다.

승인 기록: 중복 행을 지우고 대상 댐의 행을 고른다

- 원본은 한강수계 16 개 시설의 승인 3,929 행이다. 행마다 원본 행 번호와 출처 열(기관, 파일명 등)을 붙인다. 원본 행 번호는 구조화 자료 매핑에서 Entity 의 출처가 원본의 어느 행인지 가리키는 데 쓰인다.
- 그림 2 에서 2220 행은 2218 행과 값이 모두 같다. 같은 승인이 두 번 입력된 것으로 보고 2220 을 지우고, (지운 번호, 남긴 번호, 이유)를 로그에 남긴다. 2219 는 방류시작시간이 달라 남긴다. 이렇게 지운 행은 전체 7 행이다.
- 괴산댐, 팔당댐처럼 대상 4 개 댐(충주, 소양강, 횡성, 광동)이 아닌 행은 쓰지 않는다. 남은 80 행이 구조화 자료 매핑의 입력이다.

승인 기록 원본 — 한강수계 16개 시설, 3,929행 중 순차번호 2216~2222

원본 행 번호 (추가)	순차번호	관측소명	승인일	방류시작시간	접수 방류량	비고	출처 (추가)	처리
2216	2216	괴산댐	2016-07-04	2016-07-04 15:30	736	(감소방류 -295㎥/s)	…	대상 외
2217	2217	팔당댐	2016-07-04	2016-07-04 19:00	899	(초기수문방류 130㎥/s)	…	대상 외
2218	2218	광동	2016-07-04	2016-07-04 18:00	200		…	남김
2219	2219	광동	2016-07-04	2016-07-04 18:40	200		…	남김
~~2220~~	~~2220~~	~~광동~~	~~2016-07-04~~	~~2016-07-04 18:00~~	~~200~~		—	지움 = 2218
2221	2221	청평댐	2016-07-04	2016-07-04 21:30	755	(초기수문방류 99㎥/s)	…	대상 외
2222	2222	팔당댐	2016-07-04	2016-07-04 19:20	1757	(증가방류 858㎥/s)	…	대상 외

3,929행 → 중복 7행 지움 → approval_records_clean.csv 3,922행 → 대상 4개 댐 → 80행

지운 행은 approval_preprocess_log.csv에 (지운 번호 2220, 남긴 번호 2218, 이유)로 남긴다. 2219는 방류시작시간(18:40)이 달라 남긴다.

그림 2. 승인 기록 원본의 실제 행과 처리 — 중복 행 2220 을 지우고 대상 댐 행만 남긴다

K-water 시간자료: 두 파일을 이어 붙이고 24 시를 다음날 0 시로 읽는다

- 시간자료는 두 번에 나눠 받았다. 승인 기간에 맞춰 받은 파일(2010-08~2020-12)과 처음 받은 파일(2021-01~2026-09)은 기간이 이어져 겹치는 시각이 없으므로 그대로 이어 붙인다.
- 시각 라벨 SDATE_raw 는 YYYYMMDDHH 형식이고 시가 01~24 로 되어 있다. "24 시"는 일반 시각으로 읽을 수 없으므로 다음날 0 시로 읽는다. 그림 3 의 2020123124 가 2021-01-01 00:00 이 된다. 이렇게 해야 승인·운영 시각과 같은 시각 체계로 비교할 수 있다.
- 수위, 유입량 같은 값은 바꾸지 않고, 빈 값도 채우지 않는다. 결과는 4 개 댐 × 6 개 변수의 508,246 개 (댐, 시각) 이며, KG 가 아닌 측정 저장소에 둔다.

파일 ② 승인 기간에 맞춰 추가로 받은 자료 (2010-08 ~ 2020-12)

DAM_CD	SDATE_raw	수위	유입량
1003110	2020123122	131.54	9.441
1003110	2020123123	131.55	171.565

↕ 기간이 이어진다 (겹치는 시각 없음)

파일 ① 처음 받은 자료 (2021-01 ~ 2026-09)

DAM_CD	SDATE_raw	수위	유입량
1003110	2020123124	131.54	0
1003110	2021010101	131.55	186.52

⟹

측정 저장소 이어 붙이고 시각을 읽음

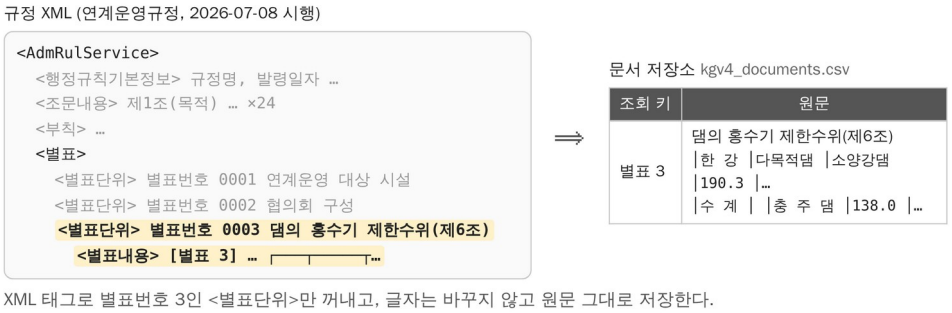
SDATE_raw	시각	수위	유입량
2020123122	2020-12-31 22:00	131.54	9.441
2020123123	2020-12-31 23:00	131.55	171.565
2020123124	2021-01-01 00:00	131.54	0
2021010101	2021-01-01 01:00	131.55	186.52

· 라벨의 시(HH)가 01~24라서 24시는 다음날 0시로 읽는다 (노란 행). 값은 바꾸지 않는다.
· 4개 댐 × 6개 변수(수위, 저수량, 강우량, 유입량, 총방류량, 저수율), 508,246개 (댐, 시각). 빈 값은 채우지 않는다.

그림 3. K-water 시간자료의 실제 행 — 두 파일의 경계와 24 시 변환 (충주댐)

연계운영규정 별표 3: XML 에서 별표 3 만 꺼낸다

- 규정 XML 에는 규정 정보, 조문 24 개, 부칙, 별표 1~3 이 태그로 나뉘어 있다(그림 4).
- 태그 <별표단위> 중 별표번호가 0003 인 것의 <별표내용>만 꺼내, 글자를 바꾸지 않고 문서 저장소에 조회 키 "별표 3"으로 저장한다. 홍수기 제한수위의 근거로 원문을 그대로 보여 주기 위해서이다.
- 원문의 문자로 그린 표에서 댐별 값을 읽는 일은 비정형 텍스트 추출(4.3)에서 한다.



XML 태그로 별표번호 3인 <별표단위>만 꺼내고, 글자는 바꾸지 않고 원문 그대로 저장한다.

그림 4. 규정 XML 의 구조와 별표 3 꺼내기

3. 구조화 자료 매핑

3.1 따른 원리: D2RQ 의 매핑 원리

- Zhou 외(2023)는 댐 안전 모니터링 온톨로지 OntoDSMS 를 만들면서, MySQL 의 모니터링 데이터를 기존 오픈소스 도구 D2RQ 로 온톨로지에 연결했다. 본 연구는 D2RQ 의 매핑 원리를 벤치마킹했다(부록 E.2).
- 원리 1. 행의 키 값으로 Entity 의 식별자를 만든다 (예: item_ID 가 7 인 행 → item/7)
- 원리 2. 한 표에서 나온 식별자는 모두 같은 Class 의 Entity 가 된다
- 원리 3. 열 하나를 Data Property 하나에 대응시키고, 셀 값을 그 값으로 옮긴다
- 원리 4. 열과 속성의 대응은 사람이 온톨로지 용어로 정한다
- 그림의 노란 점선 상자는 그 과정에 쓰인 원리와 D2RQ 구문이다. 문헌의 instance 는 우리 용어로 Entity 이다.

3.2 승인 데이터: 승인 표 한 행이 Entity 가 되는 과정

입력: 승인 표와 열의 역할 (원리 4)

- 입력: 전처리를 마친 승인 표(approval_records_clean.csv)에서 대상 4 개 댐의 80 행. 한 행은 승인 한 건이다.
- 변환하기 전에 열마다 어떤 역할을 할지 사람이 먼저 정했다. 표 2 의 머리 색이 역할이다.

표 2. 승인 표(일부)와 열의 역할

순차번호	관측소코드	관측소명	승인년월일시분	방류시작시간	접수방류량	비고
473	1006110	횡성	2011-06-24	2011-06-24 18:00	100	수문방류 승인
504	**1003110**	**충주**	**2011-06-27**	**2011-06-27 18:00**	**2000**	**(실제방류는 20 시부터)**
574	1003110	충주	2011-07-01	2011-07-01 18:00	2000	(실제 방류는 19 시부터)
식별자 키 (과정 1)	옮기지 않음 (댐 코드와 같은 값)	옮기지 않음 (코드와 같은 정보)	속성 하나 (과정 3)	세 열을 내용 속성 하나로 (과정 3)		

- 관측소코드: 자료 준비 때 승인 파일의 관측소코드와 K-water 의 댐 코드(DAM_CD)를 대조했고, 4 개 댐 모두 같은 코드였다(1003110 충주 ↔ 1003110 충주댐 등). 댐 자체의 정보는 댐 자료에서 만들므로(3.3) 승인 Entity 의 속성으로 옮기지 않는다.
- 관측소명: 관측소코드와 같은 정보라 옮기지 않는다.
- 이 밖에 원본 CSV 의 행 번호와 출처 열(기관, 내려받은 주소, 원본 파일명 등)이 있으며, 과정 4·5 에 쓴다.

과정 1. 식별자 만들기 (원리 1)

- 틀 "APR:" + {순차번호}에 504 를 넣어 승인 Entity APR:504 를 만든다.
- 같은 행에서 틀 "EVI:AP:" + {순차번호}로 그 승인이 적힌 자료의 Entity EVI:AP:504 를 만든다.

- 순차번호를 키로 쓴 이유: 원본 자료가 승인 한 건마다 매긴 번호이고, 전처리에서 중복 행을 지운 뒤에는 표 안에서 유일하다. 그래서 80 행은 서로 다른 ID 80 쌍이 된다.
- ID 는 처리 순서나 무작위 번호가 아니라 원본의 순차번호로 만든다. 그래서 같은 행은 언제 다시 만들어도 같은 ID 가 된다.



그림 5. 식별자 만들기 — 키 열의 값을 틀에 끼운다

과정 2. Class 지정 (원리 2)

- 승인 표에서 만든 APR: ID 80 개는 모두 Approval Class 의 Entity 로 기록한다.
- EVI:AP: ID 80 개는 모두 EvidenceSource Class 의 Entity 로 기록한다.

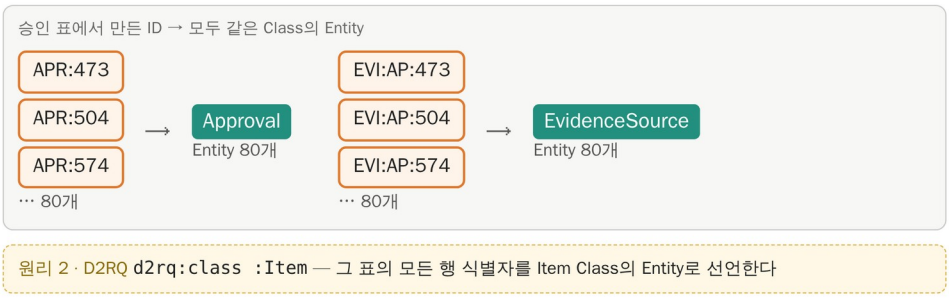


그림 6. Class 지정

결과 → kgv4_entities.csv 에 2 줄: APR:504 (Approval) · EVI:AP:504 (EvidenceSource)

과정 3. 열 → Data Property (원리 3)

- 승인년월일시분 → approvalTime. 원본 값에 시각이 없어 앞 10 글자를 날짜로 저장한다(column).
- 방류시작시간, 접수방류량, 비고 → approvalContent 하나. 세 값을 글자 틀에 넣어 저장한다(pattern). 비고는 원문 그대로 넣는다.
- 세 열을 합친 이유: 주어진 온톨로지에 Approval 의 내용 속성이 approvalContent 하나뿐이었다(5 장 검토할 점 1).

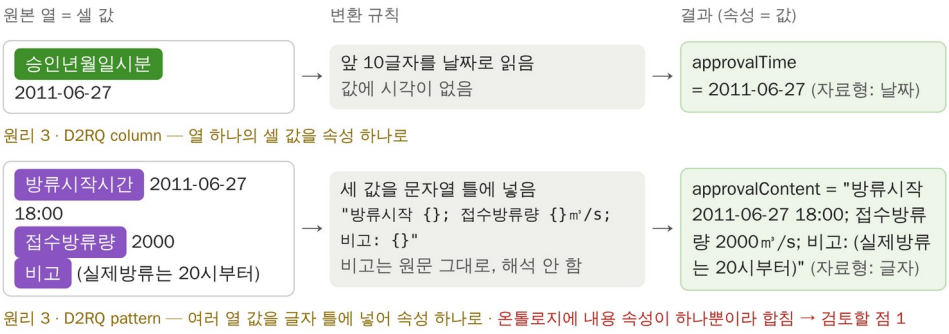


그림 7. 열 → Data Property — 원본 셀 값, 변환 규칙, 결과

결과 → kgv4_properties.csv 에 2 줄: APR:504 의 approvalTime = 2011-06-27 (날짜) · APR:504 의 approvalContent = 방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20 시부터) (글자)

과정 4. 근거 Entity 와 연결

- 같은 행의 출처 열을 근거 Entity EVI:AP:504 의 속성으로 옮긴다(아래 결과).
- APR:504 와 EVI:AP:504 는 같은 504 행을 처리하면서 동시에 만든다. 그래서 만드는 그 자리에서 APR:504 — supportedBy→ EVI:AP:504 로 잇는다(그림 8).
- 두 ID 의 504 는 둘이 같은 원본 행에서 나왔다는 표시이다. 나중에 ID 글자(AP, APR)를 맞춰 보고 찾아서 잇는 것이 아니다.



그림 8. 같은 행에서 함께 만들며 잇는다 (APR:504 → EVI:AP:504)

결과 → kgv4_properties.csv 에 3 줄: EVI:AP:504 의 sourceTitle = 환경부 한강홍수통제소 홍수예보 댐방류승인 (글자) · EVI:AP:504 의 sourceType = 승인자료 (글자) · EVI:AP:504 의 sourceLocator = 순차번호 504 (글자)

과정 5. 출처 기록

- 우리 원본은 DB 가 아니라 내려받은 CSV·JSON 파일이어서, 매핑한 값을 옮겨 저장한다.
- 값을 옮기면 KG 의 값과 원본 행의 연결이 끊긴다. 그래서 Entity 마다 원본 파일, 그 안의 위치, 적용한 규칙, 값의 성격을 출처 파일에 남긴다.
- 위치(504 번째 행)로 원 파일의 해당 행을 바로 열 수 있다. 값의 성격 "원본 그대로"는 원본 값을 가공 없이 옮겼다는 뜻이다.

결과 → kgv4_provenance.csv 에 1 줄: APR:504 ← approval_records_clean.csv, 504 번째 행, 규칙 MR-APR, 원본 그대로

결과

- 결과물은 CSV 파일 3 개(kgv4_entities, kgv4_properties, kgv4_provenance)이며, 세 파일의 줄을 Entity ID(APR:504)로 모으면 Entity 하나의 전체 모습이 된다(그림 9). 80 행 전체로는 Approval 80 개, EvidenceSource 80 개이다.

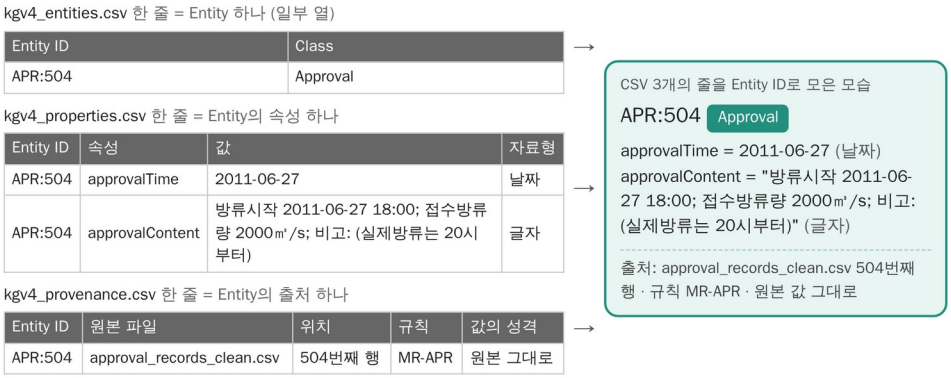


그림 9. CSV 파일 3 개의 줄과 그것을 모은 Entity (APR:504)

3.3 댐 데이터: K-water 일자료 → Dam

입력: 대상 댐 목록과 K-water 일자료

- 대상 댐 목록: 승인 자료의 관측소명(충주, 소양강, 횡성, 광동)과 표준 댐명, K-water 댐 코드(DAM_CD)를 짝지은 목록이다. 승인 파일의 관측소코드와 K-water 댐 코드는 4 개 댐 모두 같다.
- K-water 일자료: 한 행이 댐 하나의 하루 측정값 하나이고, 행마다 댐 코드, 댐 이름, 댐 유형이 반복해서 붙어 있다. 댐 한 개가 한 행인 표는 따로 없다.

과정 1. 식별자 만들기 (원리 1)

- 대상 댐 목록을 위에서부터 하나씩 꺼내 DAM_CD 를 틀 "DAM:" + {DAM_CD}에 넣는다. 충주댐은 1003110 이므로 DAM:1003110 이 된다.
- 일자료에서 충주댐의 행 12,582 개는 모두 DAM_CD 가 1003110 이다. ID 를 키 값으로 만들기 때문에 이 행들은 모두 같은 ID DAM:1003110 을 가리키고, Entity 하나로 모인다.
- 4 개 댐에서 Dam ID 4 개가 만들어진다.

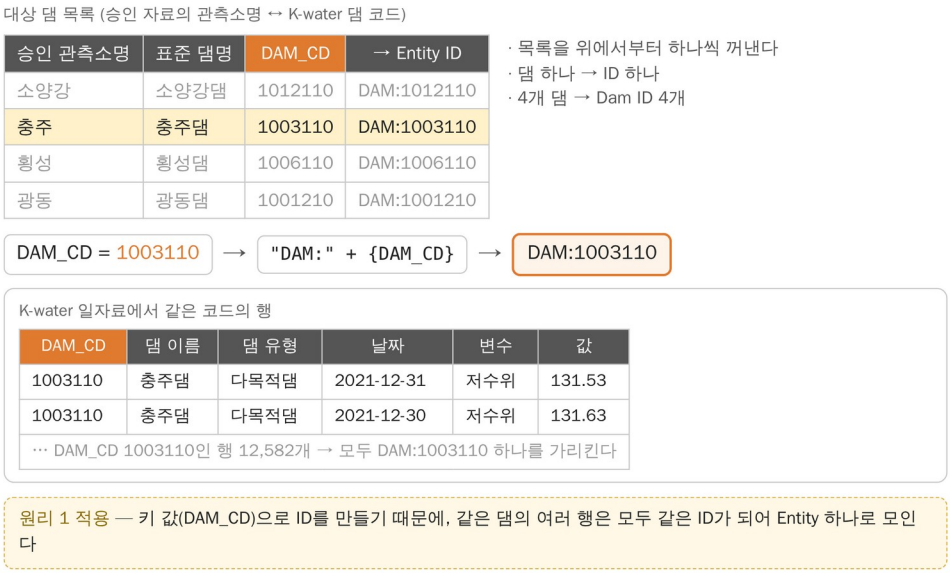


그림 10. 댐 식별자 만들기 — 대상 댐 목록의 DAM_CD 를 틀에 끼운다

과정 2. Class 지정 (원리 2)

- 대상 댐 목록에서 만든 DAM: ID 4 개는 모두 Dam Class 의 Entity 로 기록한다.

결과 → kgv4_entities.csv 에 1 줄: DAM:1003110 (Dam)

과정 3. 열 → Data Property (원리 3)

- 댐 유형 → damType. 일자료를 한 행씩 읽으며 댐 코드마다 처음 나온 댐 유형을 적어 두고, 대상 댐의 코드로 찾아 원문 그대로 저장한다. 충주댐은 다목적댐, 광동댐은 용수댐이다.
- 표준 댐명 → damName. 대상 댐 목록의 표준 댐명(충주댐)을 그대로 저장한다. 승인 자료의 "충주"와 맞춘 이름이다(원리 4).

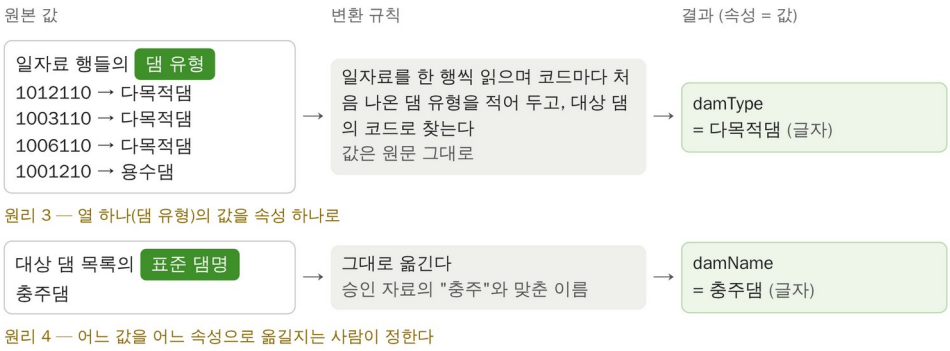


그림 11. 댐 속성 — 원본 값, 변환 규칙, 결과

결과 → kgv4_properties.csv 에 2 줄: DAM:1003110 의 damName = 충주댐 (글자) · DAM:1003110 의 damType = 다목적댐 (글자)

과정 4. 근거 Entity 와 연결

- 댐에는 근거 Entity 를 두지 않는다. 온톨로지에서 supportedBy 는 수문·기상 상태, 운영행위, 승인, 기준에만 정의되어 있다(부록 C). 댐이 어느 자료에서 왔는지는 과정 5 의 출처로 남긴다.

과정 5. 출처 기록

- 원본 파일(일자료)과 위치(DAM_CD=1003110), 규칙, 값의 성격을 남긴다. 댐 유형은 K-water 원문을 그대로 옮겼으므로 "원본 그대로"이다.

결과 → kgv4_provenance.csv 에 1 줄: DAM:1003110 ← kwater_mywater_period_daily_long.csv.gz, DAM_CD=1003110, 규칙 MR-DAM, 원본 그대로

결과

- 대상 댐 4 개에서 Dam Entity 4 개가 만들어진다. 세 파일의 줄을 Entity ID 로 모으면 그림 12 와 같다.

kgv4_entities.csv 한 줄 = Entity 하나

Entity ID	Class
DAM:1003110	Dam

kgv4_properties.csv 한 줄 = Entity의 속성 하나

Entity ID	속성	값	자료형
DAM:1003110	damName	충주댐	글자
DAM:1003110	damType	다목적댐	글자

kgv4_provenance.csv 한 줄 = Entity의 출처 하나

Entity ID	원본 파일	위치	규칙	값의 성격
DAM:1003110	kwater_mywater_period_daily_long.csv.gz	DAM_CD=1003110	MR-DAM	원본 그대로

↓ 세 파일의 줄을 Entity ID로 모으면

DAM:1003110 Dam
damName = 충주댐 (글자)
damType = 다목적댐 (글자)
출처: K-water 일자료 · DAM_CD=1003110 · 규칙 MR-DAM · 원본 그대로

그림 12. 댐 데이터의 결과 — CSV 파일 3 개의 줄과 그것을 모은 Entity (DAM:1003110)

3.4 수문·기상 데이터: K-water 시간자료 → HydrometeorologicalState

입력: K-water 시간자료와 변수 목록

- K-water 시간자료: 한 행이 (댐 코드, 시각)이고 변수가 열로 펼쳐진 넓은 형태이며, 4 개 댐 553,609 행이다.
- 변수 목록: 변수명과 그 값이 들어 있는 열 이름의 짝 6 개이다(수위 DATA1, 저수량 DATA2, 강우량 DATA3, 유입량 DATA4, 총방류량 DATA6, 저수율 DATA7). DATA5 는 원본에 변수 이름이 없어 쓰지 않는다.

과정 1. 식별자 만들기 (원리 1 응용)

- 행마다 Entity 를 만들면 55 만 개가 된다. KG 에 필요한 것은 "어느 댐에 어떤 측정 변수가 있는가"이므로, (댐, 변수)마다 Entity 하나를 만든다.
- 대상 댐을 하나씩, 각 댐에서 변수를 하나씩 꺼내 DAM_CD 와 변수명을 틀 "HMS:" + {DAM_CD} + ":" + {변수}에 넣는다. 예: HMS:1003110:수위
- 키가 키 열(DAM_CD)과 열 머리(변수명) 두 곳에 있는 넓은 표에 원리 1 을 맞춘 것이다. 4 개 댐 × 6 개 변수 = 24 개.

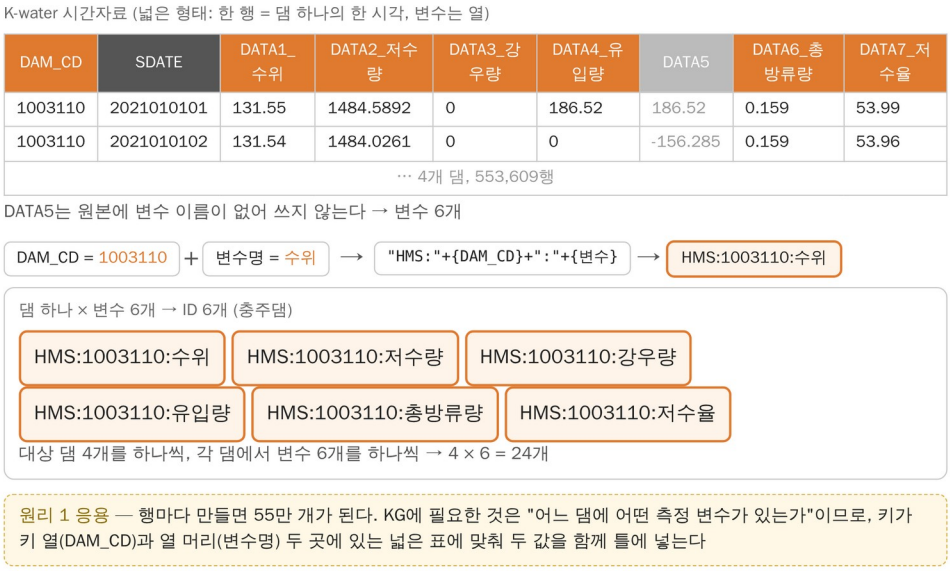


그림 13. 수문·기상 상태 식별자 만들기 — 댐 코드와 열 머리의 변수명을 틀에 끼운다

과정 2. Class 지정 (원리 2)

- 시간자료에서 만든 HMS: ID 24 개는 모두 HydrometeorologicalState Class 의 Entity 로 기록한다.

결과 → kgv4_entities.csv 에 1 줄: HMS:1003110:수위 (HydrometeorologicalState)

과정 3. 열 → Data Property (원리 3)

- 열 머리의 변수명 → variableType(수위), DAM_CD → stationCode(1003110)로 저장한다.
- 시각별 값(131.55 등)은 KG 에 넣지 않고 측정 저장소에 그대로 둔다. 질의할 때 stationCode 와 시각으로 측정 저장소에서 찾는다.

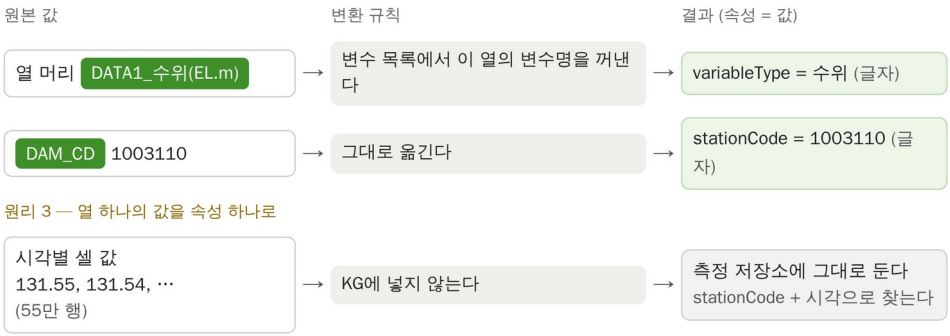


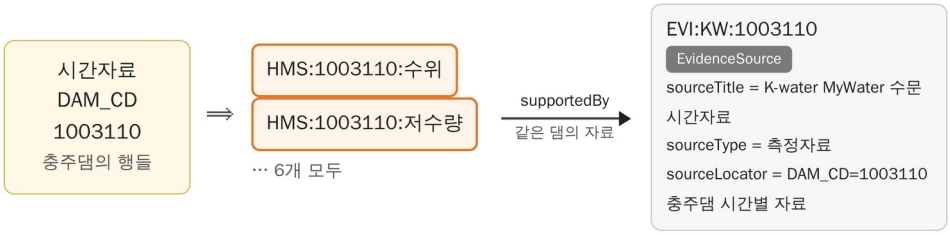
그림 14. 수문·기상 상태 속성 — 원본 값, 변환 규칙, 결과

결과 → kgv4_properties.csv 에 2 줄: HMS:1003110:수위의 variableType = 수위 (글자) · HMS:1003110:수위의 stationCode = 1003110 (글자)

과정 4. 근거 Entity 와 연결

- 댐마다 측정자료 근거 Entity 를 한 번 만든다. ID 는 "EVI:KW:" + {DAM_CD}이고, 속성은 sourceTitle, sourceType(측정자료), sourceLocator(그 댐의 시간별 자료)이다.

- 그 댐의 HMS 6 개를 만들 때마다 같은 댐 코드의 근거 Entity 에 supportedBy 로 잇는다. 같은 코드로 만들었으므로 찾을 필요가 없다.



① 댐마다 근거 Entity를 한 번 만든다(ID "EVI:KW:" + {DAM_CD}) → ② 그 댐의 HMS 6개를 만들 때마다 supportedBy로 잇는다. 같은 댐 코드로 만들었으므로 찾을 필요가 없다.

그림 15. 수문·기상 상태의 근거 연결 — 같은 댐의 측정자료 근거 Entity 에 잇는다

결과 → kgv4_properties.csv 에 3 줄: EVI:KW:1003110 의 sourceTitle = K-water MyWater 수문 시간자료 (글자) · EVI:KW:1003110 의 sourceType = 측정자료 (글자) · EVI:KW:1003110 의 sourceLocator = DAM_CD=1003110 충주댐 시간별 자료 (글자)

과정 5. 출처 기록

- 위치에는 값이 들어 있는 열 이름(DATA1_수위(EL.m))을 남긴다. 이 열 이름과 stationCode 로 측정 저장소의 시각별 값을 바로 찾을 수 있다.

결과 → kgv4_provenance.csv 에 1 줄: HMS:1003110:수위 ← kwater_mywater_hydr_H_wide.csv.gz, DATA1_수위(EL.m), 규칙 MR-HMS, 원본 그대로

결과

- 4 개 댐 × 6 개 변수로 HMS Entity 24 개와 측정자료 근거 Entity 4 개가 만들어진다(그림 16).

kgv4_entities.csv 한 줄 = Entity 하나

Entity ID	Class
HMS:1003110:수위	HydrometeorologicalState
EVI:KW:1003110	EvidenceSource

kgv4_properties.csv 한 줄 = Entity의 속성 하나

Entity ID	속성	값	자료형
HMS:1003110:수위	variableType	수위	글자
HMS:1003110:수위	stationCode	1003110	글자
EVI:KW:1003110	sourceType	측정자료	글자
EVI:KW:1003110	sourceLocator	DAM_CD=1003110 충주댐 시간별 자료	글자

kgv4_provenance.csv 한 줄 = Entity의 출처 하나

Entity ID	원본 파일	위치	규칙	값의 성격
HMS:1003110:수위	kwater_mywater_hydr_H_wide.csv.gz	DATA1_수위(EL.m)	MR-HMS	원본 그대로

↓ 세 파일의 줄을 Entity ID로 모으면

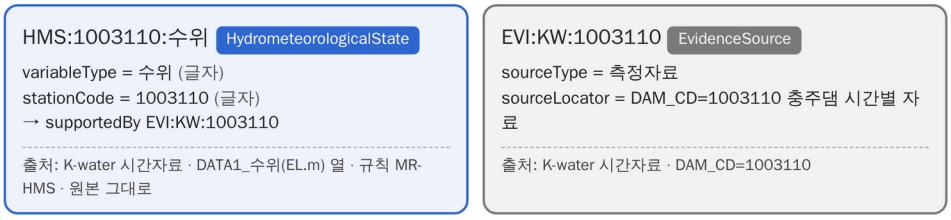
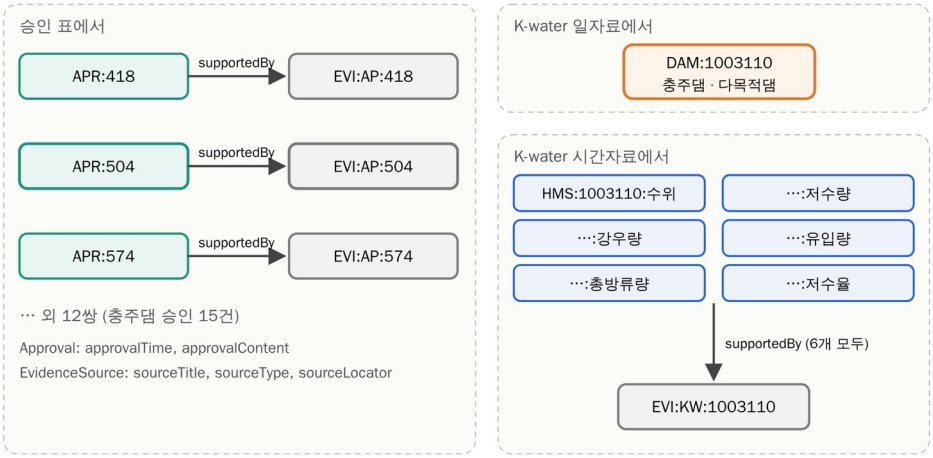


그림 16. 수문·기상 데이터의 결과 — CSV 파일 3 개의 줄과 그것을 모은 Entity (HMS:1003110:수위)

3.5 구조화 자료 매핑이 끝난 상태

- 그림 17 은 충주댐에 관련된 Entity 를 만든 자료별로 모은 것이다.
- 각 자료에서 Entity 와 속성이 만들어졌고, 같은 행·같은 자료에서 함께 만든 근거만 supportedBy 로 이어져 있다.
- 서로 다른 자료에서 만든 승인, 댐, 측정 상태 Entity 는 아직 서로 떨어져 있다. 이것을 개체 해소·연결에서 잇는다 (다음 주).

- 4 개 댐 전체: Approval 80, EvidenceSource 84(승인자료 80, 측정자료 4), HMS 24, Dam 4. 모두 Entity 192 개, 속성 468 개.



각 자료에서 Entity와 속성이 만들어졌고, 같은 행·같은 자료에서 함께 만든 근거만 supportedBy로 이어져 있다(실선). 서로 다른 자료에서 만든 승인, 댐, 측정 상태 Entity는 아직 서로 떨어져 있다.

그림 17. 구조화 자료 매핑이 끝난 상태 — 충주댐의 Entity

4. 비정형 텍스트 추출

4.1 따른 원리: Huang 외(2026)의 규칙 기반 지식 추출

- Huang 외(2026)에서 안전평가 결과(예: "홍수 방어 안전 등급은 C 등급이다")는 표준화된 문형으로 쓰이므로 딥러닝 없이 정규식으로 바로 뽑는다. 표현이 제각각인 위험 서술에만 LLM 을 쓴다(부록 E.1).
- 우리 자료는 규칙 쪽에 해당한다. 비고는 한 기관이 쓰는 짧은 문구라 어휘가 한정되어 있고("탄력적 방류", "초기수문방류", "실제방류는 N 시부터"), 별표 3 은 칸 배치가 고정된 표이다. 그래서 LLM 추출은 쓰지 않았다.
- 두 데이터 모두 먼저 규칙으로 글에서 값을 뽑아 중간 파일에 적고(추출), 그 중간 파일을 한 줄씩 읽어 구조화 자료 매핑과 같은 과정 1~5 로 Entity 를 만든다.

4.2 승인 비고 데이터 → Operation (운영행위)

입력

- 구조화 자료 매핑에서 승인 Entity 에 원문 그대로 옮긴 비고. 80 개 승인의 비고를 한 건씩 처리한다.

추출: 비고에서 행위와 시각을 뽑는다

- ① 행위가 아닌 조건 문구를 지운다: "발전방류 포함", "여수로 자연월류량 제외" 등
- ② 승인 변경 문구("변경", "연장", "정정")가 있으면 새 방류가 아니라 같은 방류의 승인을 바꾼 것으로 보고 운영행위를 만들지 않는다. 만들지 않은 사유는 기록한다.
- ③ 행위 문구 규칙(표 3)을 대조해, 잡힌 문구마다 운영행위 하나를 만든다. 한 비고에서 두 개가 잡히면 두 개를 만든다.
- ④ 행위 문구가 없으면 일반 "방류" 하나를 만든다(실제 방류 시각 문구, 일반 방류 문구, 비고 없음).
- ⑤ 시각: 비고에 시각이 적혀 있으면 그 시각, 없으면 승인의 방류시작시간을 쓴다. 504 는 승인된 18:00 대신 비고의 20:00 이다.

표 3. 행위 문구 규칙과 적용 건수 (대상 80 건)

정규식 기호: "|"는 "또는", "\s*"는 공백이 없거나 여러 칸, "\d"는 숫자 한 글자, "\d+"는 숫자 여러 글자, 괄호 "( )"로 감싼 부분은 뽑아낼 값, "\("는 괄호 글자 그 자체이다. 예: "실제\s*방류는\s*(\d+)\s*시부터"는 "실제방류는 20 시부터"와 "실제 방류는 20 시부터"를 모두 찾고 20 을 뽑는다.

operationType	찾는 문구 (정규식)	예	건수
방류종료	방류\s*종료	9.11 16:00 수문방류종료	1
초기방류	초기\s*수문\s*방류 | 초기\s*방류 | \(\s*초기\s*\d	(초기수문방류 100㎥/s)	9
점진·점증방류	점진 | 점증	이내 점증방류	1
증가방류	증가\s*방류 | \d\s*cms\s*증 | ㎥/s\s*증가 등	(증가방류 200 ㎥/s)	7
감소방류	감소\s*방류	—	0
탄력적 방류	탄력	최대 100㎥/s 이내 탄력적 방류	25
방류	실제\s*방류는\s*(\d+)\s*시부터 / 수문\s* 방류 등 / 비고 없음	(실제방류는 20 시부터)	25
(만들지 않음)	변경 | 연장 | 정정	방류기간 (당초) … (변경) …	17 건

승인 (방류시작)	비고 원문	적용한 규칙	Operation
504 충주 18:00	(실제방류는 20시부터)	행위 문구 없음 → 실제 방류 시각 문구 → 방류. 시각은 승인의 방류시작시간(18:00) 대신 비고의 실제 시작 시각 20시	OPR:504:1 방류 · 시작 2011-06-27 20:00
2754 소양강 14:00	최대 1500㎥/s(발전방류 포함)이내 탄력적 방류 (초기 1000㎥/s이내 방류)	조건 문구 지움 → 행위 문구 2개(초기, 탄력) → Operation 2개, 시각은 방류시작시간	OPR:2754:1 초기방류 OPR:2754:2 탄력적 방류 · 14:00
279 횡성 17:00	9.11 16:00 수문방류종료	행위 문구(종료) → 방류종료, 시각은 비고의 9.11 16:00	OPR:279:1 방류종료 · 09-11 16:00
418 충주 15:00	(비어 있음)	비고 없음 → 승인 자체를 방류로, 시각은 방류시작시간	OPR:418:1 방류 · 15:00
3519 충주 08-02 18:00	방류기간 (당초) 08/02 18:00 ~ 08/12 18:00 (변경) 08/02 18:00 ~ 08/16 24:00 …	승인 변경 문구 → 같은 방류의 변경으로 보고 만들지 않음 (사유 기록)	만들지 않음

~~취색 취소선~~ 지운 조건 문구 · 보라 행위 문구 · 주황 비고에 적힌 시각 · 빨강 승인 변경 문구

그림 18. 실제 비고 5 건에 추출 규칙을 적용한 결과

- 추출 결과는 중간 파일 operation_from_remarks_5dams.csv 에 적는다. 한 줄은 승인 하나의 행위 하나이고, 85 줄 = 만든 68 줄 + 만들지 않은 17 줄과 사유이다. 열은 순차번호, 비고 원문, 만듦 여부, operationType, 적용한 규칙, operationTime, 시각 출처 등이다.

과정 1. 식별자 만들기

- 중간 파일을 한 줄씩 읽어 "만들지 않음"(N) 줄은 건너뛴다.
- 순차번호마다 몇 번째 줄인지 세어(순번) 틀 "OPR:" + {순차번호} + ":" + {순번}에 넣는다. 504 는 한 줄이라 OPR:504:1, 2754 는 행위가 둘이라 OPR:2754:1(초기방류)과 OPR:2754:2(탄력적 방류)가 된다.

중간 파일 operation_from_remarks_5dams.csv (한 줄 = 승인 하나의 행위 하나, 85줄)

순차번호	비고 원문	만듦	operationType	순번 (세기)	→ Entity ID
504	(실제방류는 20시부터)	Y	방류	504의 1번째	OPR:504:1
2754	… 탄력적 방류 (초기 1000㎥/s이내 방류)	Y	초기방류	2754의 1번째	OPR:2754:1
2754	(같은 비고)	Y	탄력적 방류	2754의 2번째	OPR:2754:2
3519	방류기간 (당초) … (변경) …	N		건너뜀	(만들지 않음)

순차번호 = 504 + 순번 = 1 → "OPR:" + {순차번호} + ":" + {순번} → OPR:504:1

① 한 줄씩 읽어 만듦 = N이면 건너뛴다 → ② 순차번호마다 몇 번째 줄인지 센다 → ③ 틀에 넣는다. 한 승인에서 행위가 둘이면 :1, :2로 나뉜다.

그림 19. 운영행위 식별자 만들기 — 중간 파일의 줄에서 순차번호와 순번으로

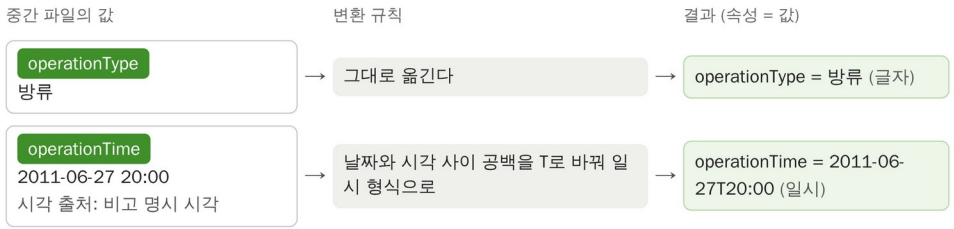
과정 2. Class 지정

- 비고에서 뽑은 행위의 OPR: ID 68 개는 모두 Operation Class 의 Entity 로 기록한다.

결과 → kgv4_entities.csv 에 1 줄: OPR:504:1 (Operation)

과정 3. 속성

- operationType 은 그대로 옮긴다(글자).
- operationTime 은 날짜와 시각 사이의 공백을 T 로 바꿔 일시 형식으로 저장한다(2011-06-27 20:00 → 2011-06-27T20:00).



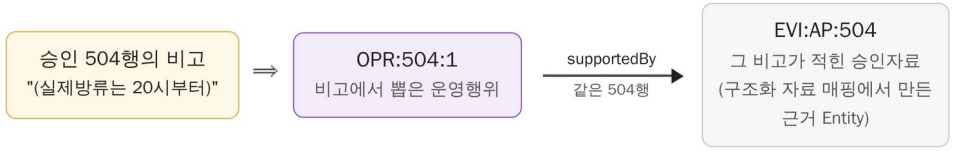
operationTime은 추출할 때 정해진다: 비고에 시각이 있으면 그 시각(504: 승인된 18:00 대신 비고의 20:00), 없으면 승인의 방류시작시간

그림 20. 운영행위 속성 — 중간 파일의 값, 변환 규칙, 결과

결과 → kgv4_properties.csv 에 2 줄: OPR:504:1 의 operationType = 방류 (글자) · OPR:504:1 의 operationTime = 2011-06-27T20:00 (일시)

과정 4. 근거 Entity 와 연결

- 비고는 승인 504 행에 적혀 있으므로, 비고에서 뽑은 운영행위의 근거도 504 행의 근거 Entity EVI:AP:504 이다(구조화 자료 매핑에서 이미 만들어 둠).
- ID 의 순차번호(504)로 바로 정해지므로 다른 표를 찾지 않고 supportedBy 로 잇는다. 새 근거 Entity 는 만들지 않는다.



비고는 승인 504행에 적혀 있으므로, 비고에서 뽑은 운영행위의 근거도 504행의 근거 Entity EVI:AP:504이다. ID의 순차번호(504)로 바로 정해지므로 찾을 필요가 없다.

그림 21. 운영행위의 근거 연결 — 비고가 적힌 504 행의 근거 Entity 에 잇는다

과정 5. 출처 기록

- 중간 파일과 순차번호, 그 줄에 적힌 적용 규칙을 남긴다.
- 값의 성격은 "비고에서 분류"이다. 원본 값을 그대로 옮긴 승인과 달리, 운영행위는 규칙으로 분류해 만든 값이기 때문이다.

결과 → kgv4_provenance.csv 에 1 줄: OPR:504:1 ← operation_from_remarks_5dams.csv, 순차번호=504, 규칙 실제 방류 시각 문구, 비고에서 분류

결과

- 80 개 승인 중 63 개에서 Operation 68 개가 만들어진다(58 개 승인에서 1 개, 5 개 승인에서 2 개). 17 개 승인은 변경 승인이라 만들지 않았다(그림 22).

kgv4_entities.csv 한 줄 = Entity 하나

Entity ID	Class
OPR:504:1	Operation

kgv4_properties.csv 한 줄 = Entity의 속성 하나

Entity ID	속성	값	자료형
OPR:504:1	operationType	방류	글자
OPR:504:1	operationTime	2011-06-27T20:00	일시

kgv4_provenance.csv 한 줄 = Entity의 출처 하나

Entity ID	원본 파일	위치	규칙	값의 성격
OPR:504:1	operation_from_remarks_5dams.csv	순차번호=504	실제 방류 시각 문구	비고에서 분류

↓ 세 파일의 줄을 Entity ID로 모으면

OPR:504:1 Operation

operationType = 방류 (글자)
operationTime = 2011-06-27T20:00 (일시)
→ supportedBy EVI:AP:504

출처: 중간 파일 순차번호=504 · 규칙 "실제 방류 시각 문구" · 비고에서 분류

그림 22. 승인 비고 데이터의 결과 — CSV 파일 3 개의 줄과 그것을 모은 Entity (OPR:504:1)

4.3 규정 별표 3 데이터 → Criterion (홍수기 제한수위)

입력

- 전처리에서 꺼내 둔 별표 3 원문. 선 문자로 그린 표라 칸이 "|"로 나뉘어 있고, 댐 이름 글자 사이에 공백이 있다("충 주 댐").

추출: 문자로 그린 표에서 댐별 칸을 읽는다

- ① 별표 3 한강수계 댐 이름(9 개)을 표에 나오는 순서대로 하나씩 꺼내, 이름 글자 사이에 공백을 허용(\s*)하는 정규식을 만든다. 예: "충주댐" → "충\s*주\s*댐"
- ② 원문에서 그 정규식으로 댐의 행을 찾고, 이어지는 두 칸(홍수기 제한수위, 시설별 최저 운영수위)의 글자를 원문 그대로 꺼낸다.
- ③ 꺼낸 순서대로 번호(CR-01, CR-02 …)를 붙여 중간 파일 criterion_list.csv 에 한 줄씩 쓴다. 번호는 별표 3 한강수계 표에 댐이 나오는 순서대로 붙인 연구용 번호이다.

문서 저장소의 별표3 원문 (일부)

```
|구 분 |댐 종류    |댐 명       |홍수기 제한수위 |시설별 최저 운영수위|
|한 강 |다목적댐   |소양강댐    |190.3           |-                   |
|수 계 |           |충 주 댐    |138.0           |-                   |
|       |           |횡 성 댐    |178.2           |-                   |
|       |           |팔 당 댐    |25.51)          |-                   |
```

댐 이름 "충주댐" → |\s*충\s*주\s*댐\s*|\s*(값)\s*|\s*(값)\s*| → 칸 원문 '138.0', '-'

글자 사이 공백을 허용(\s*)해서 "충 주 댐"과 "충주댐"을 같은 이름으로 읽는다. 칸의 글자는 해석하지 않고 원문 그대로 남긴다.

댐	제한수위 칸 원문	Criterion	criterionValue	unit
소양강댐	'190.3'	CRI:CR-01	190.3 (숫자)	EL.m
충주댐	'138.0'	CRI:CR-02	138.0 (숫자)	EL.m
횡성댐	'178.2'	CRI:CR-03	178.2 (숫자)	EL.m
팔당댐	'25.51)' (값 25.5 + 각주 1)	대상 댐 아님 · 각주가 붙은 값은 숫자만 받는 규칙에서 빠짐		

그림 23. 별표 3 의 문자로 그린 표에서 댐별 제한수위 칸 읽기

과정 1. 식별자 만들기

- 중간 파일을 한 줄씩 읽어 대상 댐이 아니거나 제한수위 칸이 숫자뿐이 아닌 줄은 건너뛴다. 팔당댐의 '25.51)'처럼 각주 표시가 붙은 칸도 건너뛴다.
- 남은 줄의 번호를 틀 "CRI:" + {번호}에 넣는다. 충주, 소양강, 횡성 3 개이며, 광동댐은 별표 3 에 없다.

중간 파일 criterion_list.csv (한 줄 = 별표3 한강수계의 댐 하나, 9줄)

번호	댐	제한수위 칸 원문	대상 댐?	숫자뿐?	→ Entity ID
CR-01	소양강댐	'190.3'	예	예	CRI:CR-01
CR-02	충주댐	'138.0'	예	예	CRI:CR-02
CR-03	횡성댐	'178.2'	예	예	CRI:CR-03
CR-04 …	화천댐 …	…	아니오		건너뜀
CR-09	팔당댐	'25.51)'	아니오	아니오 (각주)	건너뜀

번호 = CR-02 → "CRI:" + {번호} → CRI:CR-02

① 한 줄씩 읽어 대상 댐이 아니거나 제한수위 칸이 숫자뿐이 아니면 건너뛴다 → ② 번호를 틀에 넣는다. 번호는 별표3 한강수계 표에 댐이 나오는 순서대로 붙인 연구용 번호이다. 광동댐은 별표3에 없다.

그림 24. 기준 식별자 만들기 — 중간 파일의 줄을 걸러 번호를 틀에 끼운다

과정 2. Class 지정

- 별표 3 에서 뽑은 CRI: ID 3 개는 모두 Criterion Class 의 Entity 로 기록한다.

결과 → kgv4_entities.csv 에 1 줄: CRI:CR-02 (Criterion)

과정 3. 속성

- 제한수위 칸 원문 '138.0'에서 따옴표 안의 숫자를 꺼내 criterionValue(숫자)로, 단위 칸을 unit 으로, 기준 종류를 criterionType 으로 저장한다.

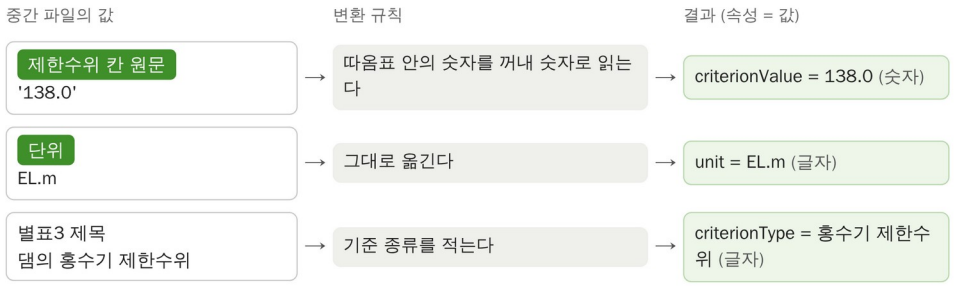
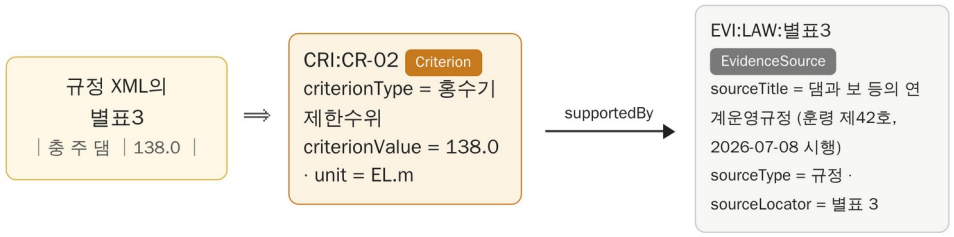


그림 25. 기준 속성 — 중간 파일의 값, 변환 규칙, 결과

결과 → kgv4_properties.csv 에 3 줄: CRI:CR-02 의 criterionType = 홍수기 제한수위 (글자) · CRI:CR-02 의 criterionValue = 138.0 (숫자) · CRI:CR-02 의 unit = EL.m (글자)

과정 4. 근거 Entity 와 연결

- 규정은 하나이므로 고정 ID "EVI:LAW:별표 3"으로 근거 Entity 를 한 번 만들고, 기준 3 개를 모두 supportedBy 로 잇는다.
- sourceLocator "별표 3"은 전처리에서 저장한 문서 저장소의 조회 키라서, 근거에서 원문으로 바로 갈 수 있다.



별표3에서 뽑은 기준값 3개(충주, 소양강, 횡성)는 모두 같은 근거 Entity EVI:LAW:별표3에 잇는다. sourceLocator "별표 3"은 문서 저장소에서 원문을 찾는 조회 키이다.

그림 26. 기준의 근거 연결 — 규정 근거 Entity EVI:LAW:별표 3 에 잇는다

결과 → kgv4_properties.csv 에 3 줄: EVI:LAW:별표 3 의 sourceTitle = 댐과 보 등의 연계운영규정 (기후에너지환경부 훈령 제 42 호, 시행 2026-07-08) (글자) · EVI:LAW:별표 3 의 sourceType = 규정 (글자) · EVI:LAW:별표 3 의 sourceLocator = 별표 3 (글자)

과정 5. 출처 기록

- 중간 파일과 번호(CR-02)를 남긴다. 칸 원문의 숫자를 그대로 옮겼으므로 값의 성격은 "원본 그대로"이다.

결과 → kgv4_provenance.csv 에 1 줄: CRI:CR-02 ← criterion_list.csv, CR-02, 규칙 MR-CRI, 원본 그대로

결과

- Criterion 3 개와 규정 근거 Entity 1 개가 만들어진다(그림 27).



그림 27. 규정 별표 3 데이터의 결과 — CSV 파일 3 개의 줄과 그것을 모은 Entity (CRI:CR-02)

4.4 비정형 텍스트 추출이 끝난 상태

- 비정형 텍스트 추출에서 Operation 68 개, Criterion 3 개, 규정 근거 EvidenceSource 1 개와 속성 148 개가 더해진다.
- 구조화 자료 매핑의 결과와 합치면 KG 에 들어갈 Entity 264 개와 속성 616 개가 모두 만들어진다. 같은 행·같은 자료에서 함께 만든 근거만 이어져 있고, 승인과 댐, 운영행위와 댐, 기준과 댐처럼 서로 다른 자료에서 나온 Entity 는 아직 이어지지 않았다. 이들을 관계로 이어 KG 를 완성하는 것이 개체 해소·연결이다(다음 주).

5. 검토할 점

1. approvalContent 하나에 세 열이 들어가 있음 (구조화 자료 매핑, 온톨로지 발전 후보)

- 현재: 방류시작시간, 접수방류량, 비고를 글자 하나로 저장한다. 접수일자 열은 어느 속성에도 들어가지 않았다.
- 이유: 구축 당시 주어진 온톨로지에서 Approval 의 Data Property 는 approvalTime 과 approvalContent 두 개뿐이었고, 온톨로지는 수정하지 않는다는 전제로 매핑했다.
- 원리 3 대로라면 속성도 따로여야 한다. 합친 결과 "접수방류량 2000㎥/s 이상인 승인", "승인일과 방류 시작의 시간 차" 같은 조건 질의를 할 수 없다.
- 선택지: ① Approval 에 dischargeStartTime(일시), requestedDischarge(숫자, ㎥/s), remark(글자), 필요하면 receiptDate(날짜)를 추가 ② 현재대로 두고 승인 내용은 답변에 원문으로만 보여 줌
- 판단 기준: 지금의 CQ4 는 댐과 기간으로 승인을 찾아 내용을 그대로 돌려주므로 분리가 꼭 필요하지 않다. 방류량이나 시각을 조건으로 쓰는 질문을 CQ 에 넣을 때 함께 정한다.

2. ID 접두어 표기 (구조화 자료 매핑, 10/19 질문: AP 와 APR)

- 연결: APR:504 와 EVI:AP:504 는 같은 504 행에서 동시에 만들어지고 그 자리에서 이어진다(과정 4, 그림 8). 그래서 글자가 달라도 연결에는 문제가 없다.
- 표기: APR 은 승인(Approval), EVI 는 근거 자료(EvidenceSource)이고, EVI 뒤의 AP 는 출처 종류(승인자료)이다. 측정자료는 KW, 규정은 LAW 이다.
- 검토: 사람이 볼 때 AP 와 APR 이 비슷해 헷갈린다. 출처 종류 표기를 Class 접두어와 맞출지(EVI:APR:504) 정한다.

3. 운영행위 자료가 없어 승인 비고로 운영행위를 구성함 (비정형 텍스트 추출, 자료의 문제)

- 자료의 문제: 실제 운영 기록(언제 수문을 열고 닫았는지, 방류를 언제 시작·종료했는지)을 대상 댐에 대해 얻지 못했다. K-water 수문 방류정보 OpenAPI 는 인증키와 심의 승인이 필요했고, 수문 개폐 자료는 다른 수계(섬진강댐)나 다른 기간(2024 년)만 공개되어 있었으며, 한강홍수통제소·WAMIS 사이트는 접속되지 않았다.
- 그래서 운영행위(CQ3)를 채우기 위해, 승인 기록의 비고에 적힌 문구로 운영행위를 구성해 보았다. 즉 Operation 은 실제 운영 기록이 아니라 승인 문서에서 읽어 낸 것이다.
- 그 결과 생기는 문제: 비고에는 승인 조건(허가한 범위와 방식, 예: "최대 X 이내 탄력적 방류"), 승인 변경, 실제 실행 정보(예: "실제방류는 20 시부터", "수문방류종료")가 섞여 있는데, 지금 규칙은 승인 변경만 구분한다. "탄력적 방류" 25 건은 허가한 방식이고, 비고가 빈 15 건은 승인 자체를 방류로 만든 것이다. 그래서 Operation 은 "실제로 한 행위"가 아니라 "승인이 허가한 행위"에 가깝다.
- 검토: 실제 운영 기록을 얻는 방법(OpenAPI 인증 신청, 기관 요청)을 찾고, 그 전까지는 비고를 승인 조건 / 승인 변경 / 실행 정보로 나누어 실행 정보만 Operation 으로, 승인 조건은 Approval 의 속성으로 옮길지 검토한다(검토할 점 1 과 연결).

6. 다음 주에 구체화할 것

- 수행은 모두 끝난 상태이므로, 다음 주에는 아래 단계의 방법론을 이번과 같은 방식(따른 원리 → 실제 행으로 본 과정 → 결과물)으로 구체화한다.

개체 해소·연결

- 서로 다르게 적힌 같은 대상(충주, 충주댐, 충 주 댐)을 기준 식별자(K-water 댐 코드)에 대응시켜 잇는 원리와, 관계마다 끝점을 무엇으로 찾는지를 구체화한다. 10/19 질문 "연결이 실제로 되나"에 대한 답이 된다.

검증

- 무엇을 검증하는지(온톨로지인지 KG 인지)부터 다시 정하고, 제약의 정의와 그 제약을 고른 근거, 판정 방법을 재정립한다. 10/19 에 가장 약하다고 지적받은 부분이다.

저장

- 결과물 CSV 가 어떤 형태로 저장되고, Cypher 를 거쳐 Neo4j 에 어떻게 들어가는지를 구체화한다.

검색과 답변

- 의도별 경로를 미리 정해 두는 지금 방식을 유지할지, 어디까지 탐색할지를 LLM 이 판단하게 할지 등 방법 자체를 다시 정하는 것을 위주로 한다.

부록 A. 용어 정리

용어	본 연구에서의 정의
Ontology	댐 운영 분야에 어떤 종류의 대상이 있고 어떤 관계·속성으로 표현하는지 정리한 의미 구조. 실제 운영데이터는 포함하지 않는다.
Class	Ontology 가 표현하는 대상의 종류. 예: Dam, Operation, Approval.
Entity	Class 에 속하는 실제 개체. 예: 충주댐은 Dam Class 의 Entity. 문헌의 instance(인스턴스)에 해당한다.

용어	본 연구에서의 정의
Property	Entity 를 기술하는 방법. Entity 와 Entity 를 연결하는 Relation(Object Property)과, Entity 에 값을 부여하는 Data Property 의 두 종류로 구분한다.
Relation (Object Property)	Property 중 대상과 대상 사이의 의미적 연결을 표현하는 관계. Ontology 에서는 어떤 종류의 대상끼리 어떤 관계를 가질 수 있는지 정의하고, 실제 KG 에서는 해당 Entity 사이에 그 관계가 적용된다.
Data Property	Property 중 대상이 가질 수 있는 값을 표현하는 속성. Ontology 에서는 각 Class 가 어떤 값을 가질 수 있는지 정의하고, 실제 KG 에서는 Entity 에 구체적인 값이 채워진다.
Knowledge Graph (KG)	Ontology 에 실제 Entity·Relation·값을 채워 만든 지식 구조. 본 연구에서는 모든 원자료를 담는 데이터베이스가 아니라, 핵심 의미·관계·근거와 외부 자료를 연결하는 중심 구조로 사용한다.
Competency Question (CQ)	Ontology 가 답할 수 있어야 하는 질문으로, 연구 범위와 필요한 지식 구조를 정하는 기준.
Hybrid Retrieval	질문에 따라 KG, 문서, 측정자료 등 서로 다른 출처에서 필요한 근거를 검색하여 결합하는 방식.
정규식	글자의 모양을 패턴으로 적어 문장에서 맞는 부분을 찾는 표현. 예: "실제\s*방류는\s*(\d+)\s*시부터"는 "실제방류는 20 시부터"에서 20 을 찾는다.

부록 B. 연구 목적과 Ontology 가 답해야 할 질문

연구 목적	현재 댐의 수문·기상 상태와 관련된 과거 운영행위, 승인, 운영기준 및 근거자료를 연결하여 댐 운영자의 의사결정을 지원한다.

전체 질문	특정 댐의 현재 수문·기상 상태와 현행 운영기준을 확인하고, 선택된 과거 시점 또는 사례의 수문·기상 상태, 실제 운영행위, 방류 승인내용 및 관련 근거자료를 함께 추적하여 현재 운영 판단을 지원할 수 있는가?

번호	질문 대상	세부 질문
CQ1	현재 수문·기상 상태	대상 댐의 현재 또는 기준시점의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 무엇인가?
CQ2	과거 수문·기상 상태	선택된 과거 시점 또는 사례에서 대상 댐의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 어떠했는가?
CQ3	과거 운영행위	해당 과거 시점 또는 사례에서 실제로 수행된 댐 운영행위는 무엇이었는가?
CQ4	과거 방류 승인	해당 과거 시점 또는 사례에서 어떤 방류 승인이 있었으며, 그 승인내용은 무엇인가?
CQ5	현재 운영기준	현재 대상 댐에 적용되는 운영기준은 무엇이며, 현재 운영 판단과 관련된 기준값은 무엇인가?
CQ6	공식 근거	각 수문·기상 상태, 운영행위, 승인내용 및 운영기준을 확인할 수 있는 공식 자료 또는 근거문서는 무엇인가?

부록 C. 온톨로지 설계

Class (6 개)

Class	의미	예	표현하는 CQ
Dam	댐 운영 의사결정의 대상이 되는 물리적 시설	충주댐, 소양강댐	CQ1~CQ6 전체의 중심 대상
HydrometeorologicalState	댐과 연결되는 수문·기상 상태(강우, 수위, 유입량, 방류량 등) 변수 정보	충주댐_강우량	CQ1·CQ2
Operation	실제로 수행된 댐 운영행위	방류, 수문 개방	CQ3 (승인과 구분)
Approval	운영행위(방류/수문 운영 등)에 대한 승인	방류 승인	CQ4

Class	의미	예	표현하는 CQ
Criterion	특정 Dam 에 적용되는 운영기준	홍수기 제한수위	CQ5
EvidenceSource	상태·운영·승인·기준을 확인할 수 있는 공식 근거자료	규정, 승인자료, 측정자료	CQ6

Relation (9 개)

Subject	Relation	Object	의미
Dam	hasHydrometeorologicalState	HydrometeorologicalState	댐이 해당 수문·기상 변수 정보와 연결됨
Operation	performedOnDam	Dam	운영행위가 특정 Dam 에서 수행됨
Approval	authorizes	Operation	Approval 이 특정 Operation 을 승인함
Approval	concernsDam	Dam	Approval 이 특정 Dam 에 관한 것임 (KG 구축 중 CQ4 에 답하기 위해 추가)
Criterion	appliesToDam	Dam	운영기준이 특정 Dam 에 적용됨
HydrometeorologicalState	supportedBy	EvidenceSource	상태를 확인할 수 있는 근거자료
Operation	supportedBy	EvidenceSource	운영행위를 확인할 수 있는 근거자료
Approval	supportedBy	EvidenceSource	승인을 확인할 수 있는 근거자료
Criterion	supportedBy	EvidenceSource	운영기준을 확인할 수 있는 근거자료

Data Property

Class	Data Property	필요한 이유
Dam	damName, damType	대상 댐을 식별하고 유형을 설명하기 위해
HydrometeorologicalState	variableType, stationCode	수문·기상 변수를 구분하고 측정자료를 찾아가기 위해
Operation	operationType, operationTime	어떤 운영이 언제 수행되었는지 확인하기 위해
Approval	approvalTime, approvalContent	언제 어떤 승인이 있었는지 확인하기 위해
Criterion	criterionType, criterionValue, unit	어떤 기준이 얼마의 값으로 적용되는지 확인하기 위해
EvidenceSource	sourceTitle, sourceType, sourceLocator	근거자료와 자료 내 근거 위치를 확인하기 위해

부록 D. KG 와 외부 저장소, 구축 대상 자료

구분	저장하는 것	예
KG	의미와 관계, 근거 위치, 외부 자료를 찾아가는 정보	승인·운영행위 Entity, 변수 종류와 측정소 코드, 근거자료의 위치
측정자료 저장소	시간별 실제 측정 시계열 값 전체	수위, 유입량, 방류량 등
문서 저장소	규정 원문	연계운영규정 별표 3 (현행)

KG 의 역할	KG 는 모든 데이터를 직접 저장하지 않고, 댐 운영에 필요한 상태·운영·승인·기준·근거의 의미 관계를 표현하며, 상세 문서와 대량 측정자료를 찾아갈 수 있도록 연결하는 중심 구조이다.

구축 대상 자료

자료	출처	범위	사용
방류 승인 기록	한강홍수통제소 댐방류승인(공공데이터포털)	충주·소양강·횡성·광동 4 개 댐 80 건 (2010-09 ~ 2021-07)	Approval, Operation
수문·기상 시간자료	K-water MyWater	4 개 댐 × 6 개 변수, 508,246 개 (댐, 시각). 충주·횡성 2010-08~, 소양강 2011-06~, 광동 2016-06~, 모두 2026-09 까지	측정자료 저장소, HMS
댐 일자료	K-water MyWater	4 개 댐의 댐 코드, 댐 이름, 댐 유형	Dam
홍수기 제한수위	연계운영규정 별표 3(법제처 XML)	충주 138.0, 소양강 190.3, 횡성 178.2 EL.m (광동댐은 별표 3 에 없음)	Criterion, 문서 저장소

부록 E. 근거 문헌의 원리

E.1 Huang 외(2026) — 소스 구분, 비정형 텍스트 추출

- 무엇인가: 중국의 결함·위험 댐 22,076 개의 안전 감정 보고서와 저수지 운영관리 플랫폼 자료로 댐 결함 지식그래프(DDKG, Dam Defects Knowledge Graph)를 만든 논문이다. 온톨로지 DDO(Dam Defects Ontology)의 최상위 Class 는 7 개다: c1 Dam, c2 Properties, c3 Location, c4 Parameter, c5 HazardForm(위험 형태), c6 HazardComponent(위험 부위), c7 SafetyEvaluation(안전평가).
- 지식 출처와 전처리: 플랫폼의 정형 표가 c1~c4 의 데이터를 대부분 채우고, 보고서의 비정형 텍스트(PDF, DOC, PNG)가 c5~c7 을 채운다. 비정형 텍스트는 마크다운으로 바꾸고(이미지는 OCR), 정규식으로 필요한 텍스트 구간의 경계를 찾아 잘라낸 뒤 하나의 말뭉치로 합친다. → 우리는 출처 구분만 따랐고, 전처리 기술은 쓰지 않았다.
- 규칙 기반 지식 추출: c7 안전평가 결과(홍수 방어, 침투, 구조, 내진, 금속 구조 각각 A/B/C 등급)는 고도로 표준화된 문형으로 쓰이므로 정규식으로 바로 뽑는다. 예: "홍수 방어 안전 등급은 C 등급이다" → C. → 비정형 텍스트 추출에서 따랐다.
- LLM 기반 지식 추출: 표현이 제각각인 위험 서술(c5, c6)은 미세조정한 LLM, 온도를 바꾼 여러 번의 추론과 투표, 온톨로지 제약 검사, 전문가 검토로 분류한다(OHCF). → 우리 자료에는 이런 서술이 없어 쓰지 않았다.
- 저장: 추출한 지식을 JSON 으로 모은 뒤 Python 스크립트로 Cypher 문을 만들어 Neo4j 에 넣는다. → 저장(다음 주)에서 다룬다.

E.2 Zhou 외(2023) — 구조화 자료 매핑

- 표기: Zhou 외(2023)는 댐 안전 모니터링을 위한 BIM·온톨로지 기반 지식관리 논문이다. OntoDSMS 는 이 논문이 만든 댐 안전 모니터링 시스템(DSMS) 도메인 온톨로지의 이름이다. D2RQ 는 이 논문이 만든 것이 아니라 기존 오픈소스 플랫폼이며, 논문은 MySQL 의 모니터링 데이터를 OntoDSMS 에 연결할 때 D2RQ 를 사용했다.
- D2RQ 는 "표는 Class, 열은 Data Property, 행은 Entity, 셀은 값"이라는 대응을 매핑 파일(Turtle 문법)의 선언으로 구현한다.
- 원리 1 (uriPattern): 식별자의 틀을 "item/@@item.item_ID@@"처럼 정하고, @@ 사이 열의 값을 행마다 끼워 넣는다. item_ID 가 7 인 행 → item/7. 같은 행은 언제나 같은 식별자를 받는다.
- 원리 2 (d2rq:class): "d2rq:class :Item"으로 그 표의 모든 행 식별자를 Item Class 의 Entity 로 선언한다. 논문: "Converting a table in a database to a class in an ontology"
- 원리 3 (PropertyBridge): 열마다 어느 Entity 에 붙일지(belongsToClassMap), 어떤 속성인지(property), 어느 열의 값을 옮길지(column)를 선언한다. 예: column "item.Genre" → 속성 :itemName. 고정 문자열과 열 값을 조합할 때는 pattern 을 쓴다. 논문: "Converting a column in a table to a DatatypeProperty in an ontology"
- 원리 4: 매핑 파일은 DB 구조로부터 자동 생성되며 이름은 DB 의 표·열 이름 그대로이다. 이를 사람이 온톨로지 용어에 맞게 고친다 (Genre 열 → :itemName).

부록 F. 참고문헌

- Huang, J., Hou, W., Zhang, S., Du, Z., & Yin, J. (2026). Dam defects ontology and knowledge graph construction from multi-source hazard data using large language models. Advanced Engineering Informatics, 76, 104987.
- Zhou, Y., Bao, T., Shu, X., Li, Y., & Li, Y. (2023). BIM and ontology-based knowledge management for dam safety monitoring. Automation in Construction, 145, 104649.
- Purohit, S., Chin, G., Mackey, P. S., & Cottam, J. A. (2024). GraphAide: Advanced Graph-Assisted Query and Reasoning System. IEEE BigData.